



C Programming

Presenter

Usha Rani Sargurunathan

Project Engineer

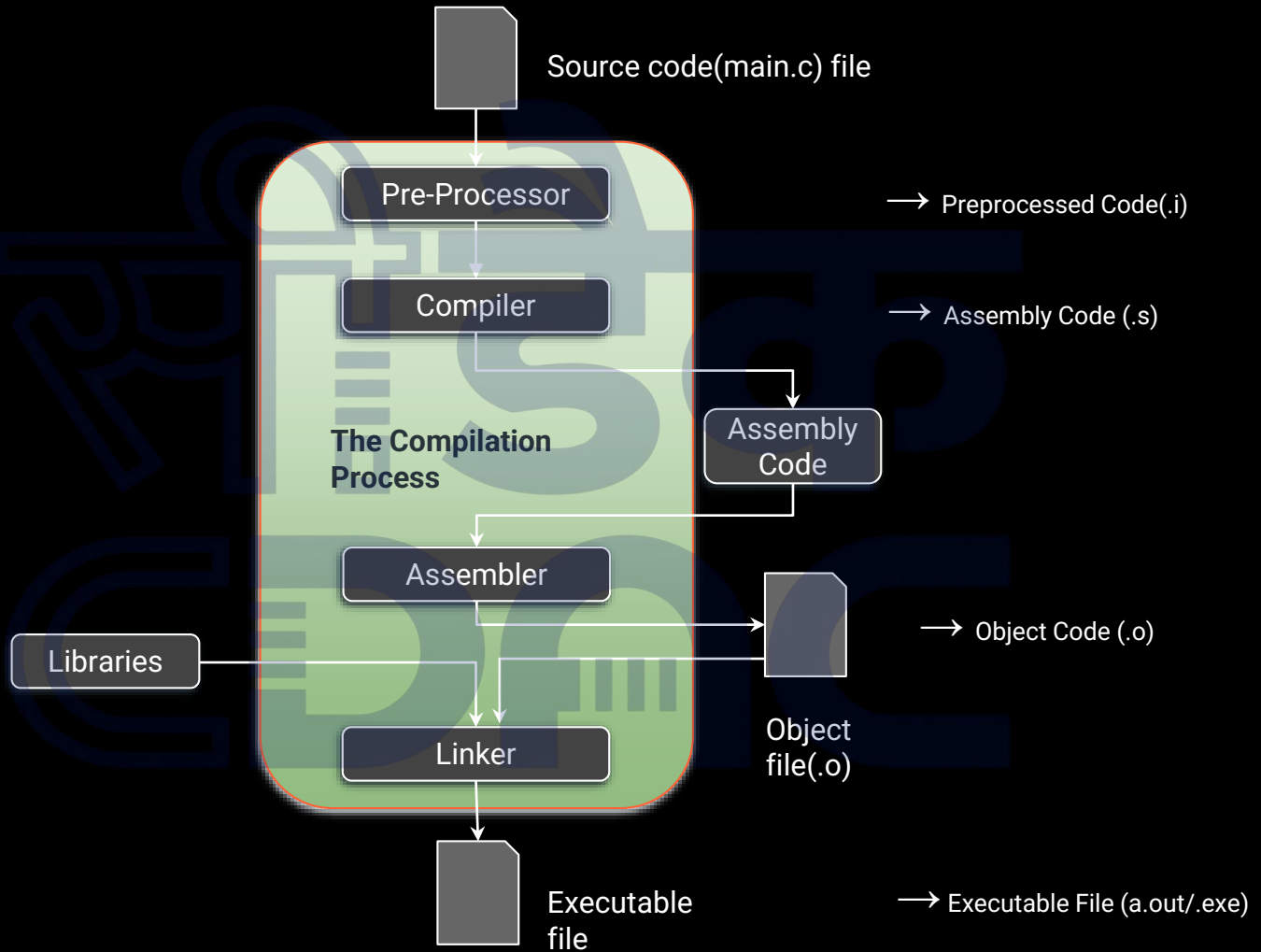
Software Technology Group

Compilation Process In C

Compilation process in C

The compilation process in C is the procedure that converts a high-level C program into an executable file that a computer can run.





Steps in the compilation process

- **Source Code** : The process begins with the source code written in a .c file by the programmer. This code is human-readable and contains C programming language syntax.
- **Preprocessing**: Handles all macros and header file inclusion.
- **Compilation**: Converts human-readable code into an intermediate assembly language.
- **Assembly**: Translates assembly code into binary object code.
- **Linking**: Combines all components into a runnable executable.

Basic Structure of a C Program

```
#include <stdio.h> // Preprocessor directive
```

```
// Function declaration (optional)
```

```
void myFunction();
```

```
// Main function: Entry point of the program
```

```
int main() {
```

```
    // Code inside main
```

```
    printf("Hello, World!\n"); // Print output
```

```
    return 0; // Indicate successful execution
```

```
}
```

```
// Function definition (if declared earlier)
```

```
void myFunction() {
```

```
    // Code inside the function
```

```
}
```



Components of a C Program

1. Preprocessor Directives:

- Begin with # (e.g., #include or #define).
- Used to include libraries (#include <stdio.h>) or define constants (#define PI 3.14).

2. Functions:

- A C program consists of functions.
- The program execution starts with the main() function.
- Other functions can be defined and called as needed.

3. Variables and Declarations:

- Variables must be declared with a data type (e.g., int, float) before use.
- Example: `int age = 25;`

4. Statements and Expressions:

- Perform operations and computations.
- Example: `a = b + c;`

5. Return Statement:

- `return 0;` indicates the successful completion of the `main()` function.
- For other functions, the return statement specifies the value to return.

Detailed Syntax

```
#include <stdio.h>           // Include standard input/output library
```

```
// Function prototype (optional but recommended for clarity)
```

```
void greetUser();
```

```
int main() {
```

```
    int number;    // Variable declaration
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &number); // Input from the user
```

```
    printf("You entered: %d\n", number); // Output the number
```

```
    greetUser();    // Call the function
```

```
    return 0;       // Exit the program
```

```
}
```

```
// Function definition
```

```
void greetUser() {
```

```
    printf("Hello, User!\n");
```

```
}
```

Explanation of Example Program

1. `#include <stdio.h>:`

- Includes the Standard Input/Output library for functions like `printf` and `scanf`.

2. `int main():`

- Entry point of the program where execution starts.
- All executable code is inside this function.

3. `printf` and `scanf`:

- `printf` is used for output.
- `scanf` is used for input.

4. Function Definition:

- The function `greetUser` is defined after `main`.
- It is called inside `main` to display a message

Structure Summary

- Preprocessor Directives: `#include <library>`
- Global Variables or Function Declarations (optional)
- `main()` Function:
 - Variable Declarations
 - Input/Output Statements
 - Logic/Operations
- Other Functions (optional)

printf() and scanf()

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

```
// #include<stdio.h>

int main(){

int number;

printf("enter a number:");

scanf("%d",&number);

printf("cube of number is:%d ",number*number*number);

return 0;

}
```

Output :
enter a number:5
cube of number is:125

printf()

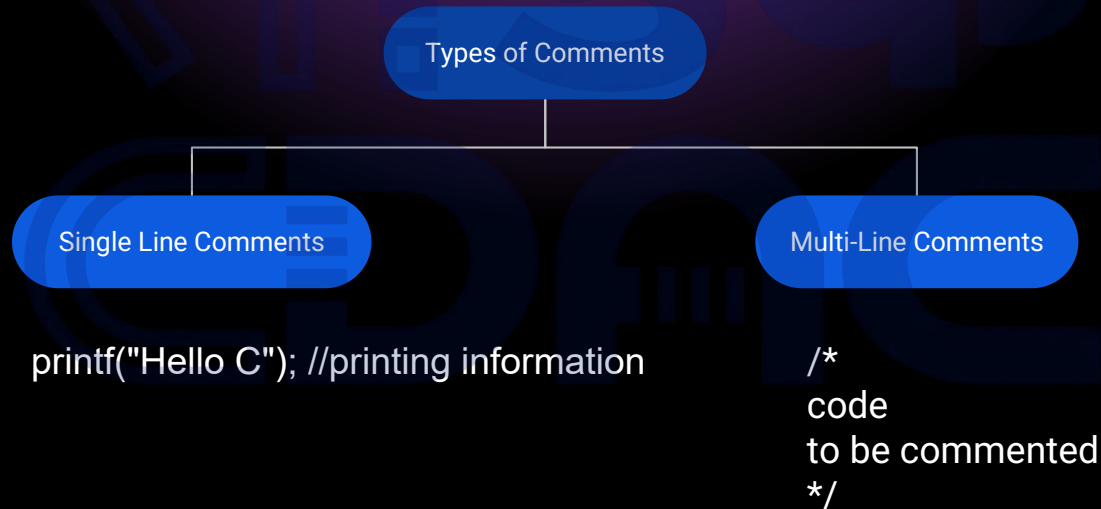
- Arbitrary strings and variable values can be printed using printf() function.
- Examples: - %d - %lf - %c - %f
- printf("Hello"); • printf("%d", roll_number);
- printf("%d %lf %c", number, basic_salary, letter);
- printf("Book price is %lf", price);

scanf()

- `#include <stdio.h>` -- function declaration
- `scanf()`
 - Used to input values from user.
 - Same format specifiers as of `printf()`.
 - Do not use any char other than format specifiers in format string.
 - To skip a char from input use `%*c`.

Comments in C

Comments in C language are used to provide information about lines of code. It is widely used for documenting code.



Tokens in C



01

Keywords

02

Identifiers

03

Constants

04

Strings

05

Special Symbols

06

Operators

→ Smallest individual unit of the program is called as token.

→ C program is made up of functions.

→ Function is made up of statements.

→ Statement contain multiple tokens

→ For example : we cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C.

Keywords

- ❖ Keywords are **predefined words** used in program, which have special meanings to the compiler.
- ❖ They are reserved words, so cannot be used as identifier.
- ❖ A list of 32 keywords in the c language is given below:

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

Identifiers

- C identifiers represent the name in the C program, for example, variables, functions, arrays, structures, unions, labels, etc.
- Rules of Identifiers:
 - ⇒ Should start with alphabet or with _ (underscore)
 - ⇒ The first character of an identifier cannot be digit it should be letter (either uppercase or lowercase)
 - ⇒ Can include alphabets, _ (underscore), digits
 - ⇒ Case sensitive

Identifiers

- **Valid C Identifiers**

age, Age, AGE, average_age, __temp,
address1, phone_no_personal, _my_name

- **Invalid C Identifiers**

Average-age, my name, \$age, #phone,
1mg, phy+maths

Format Specifier

⇒ The Format specifier is a string used in the formatted input and output functions.

⇒ The format string determines the format of the input and output



char - %c	int - %d,%i
float - %f	double - %lf
long int - %ld	short int - %hd
string type - %s	Pointer type - %p
unsigned long - %lu	unsigned short - %hu

Variables, Constants and Literals in C

Variables in C

- A variable is nothing but a *name given to a storage area* that our programs can manipulate.

- `data_type variable_name;`

- `data_type`: It represents the type of data the variable can hold.
- `variable_name`: It is the identifier for the variable, i.e., the name you give to the variable to access its value later in the program.

```
#include<stdio.h>
```

```
int main(){
```

```
int variable_name;
```

```
int count = 0;
```

```
int a=20, b=30;
```

```
//declaring 2 variable of  
integer type
```

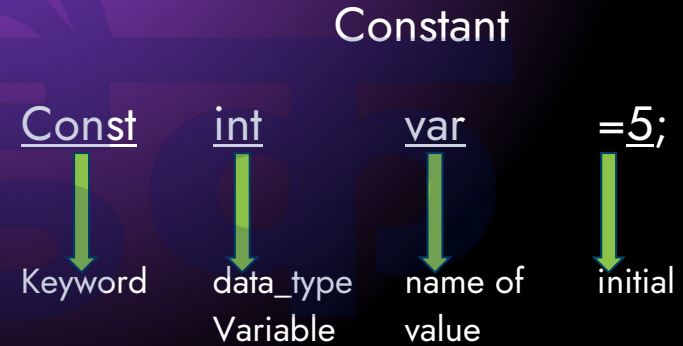
```
}
```

Constants

- The constants in C are the **read-only variables** whose values cannot be modified once they are declared in the C program.
- A constant in C can be of any of the data types.
- Defining Constant using #define Preprocessor
- Syntax to Define Constant

`const data_type var_name = value;`

`#define const_name value`



Literals

Literals are the constant values that are assigned to the variables. Literals represent fixed values that cannot be modified. Literals contain memory but they do not have references as variables.

Example:

```
int x = 5; // Here, '5' is a literal.
```

In the statement `int x = 5;` the literal 5 is used directly.

It exists only temporarily to assign the value to x.

You cannot access the memory of 5 directly since it has no name or reference.

Types of Literals

Literals in C are of different types:

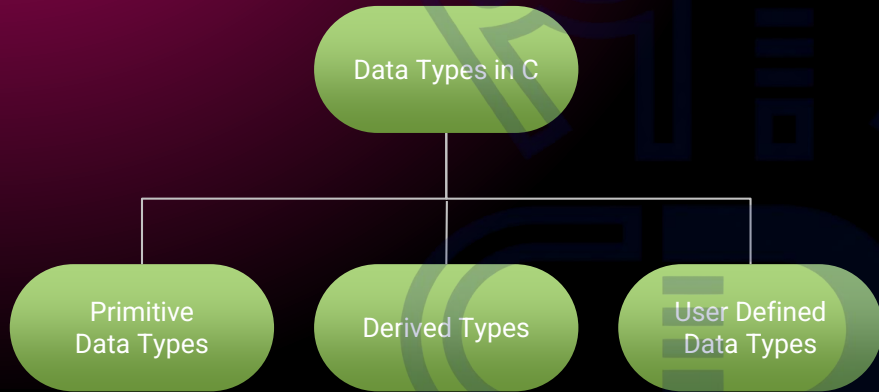
1. Integer Literals:
 - Example: 5, 10, -3.
2. Floating-point Literals:
 - Example: 3.14, 0.01.
3. Character Literals:
 - Example: 'A', 'Z'.
4. String Literals:
 - Example: "HelloWorld".

Escape sequences

• \'	Single quotation mark	prints '
• \"	Prints Double quotation mark	prints "
• \\	Backslash Character	prints \
• \a	Alert	Alerts by Generating beep
• \b	Backspace	Moves cursor one position to the left of its current position
• \f	Form Feed	Moves cursor to the beginning of next page
• \n	New Line	Moves cursor to the beginning of next line
• \r	Carriage return	Moves the cursor to the beginning of current line
• \t	Horizontal tab	Moves the cursor to next horizontal tab stop
• \v	Vertical tab	Vertical tab

Data Types In C

Data Types in C

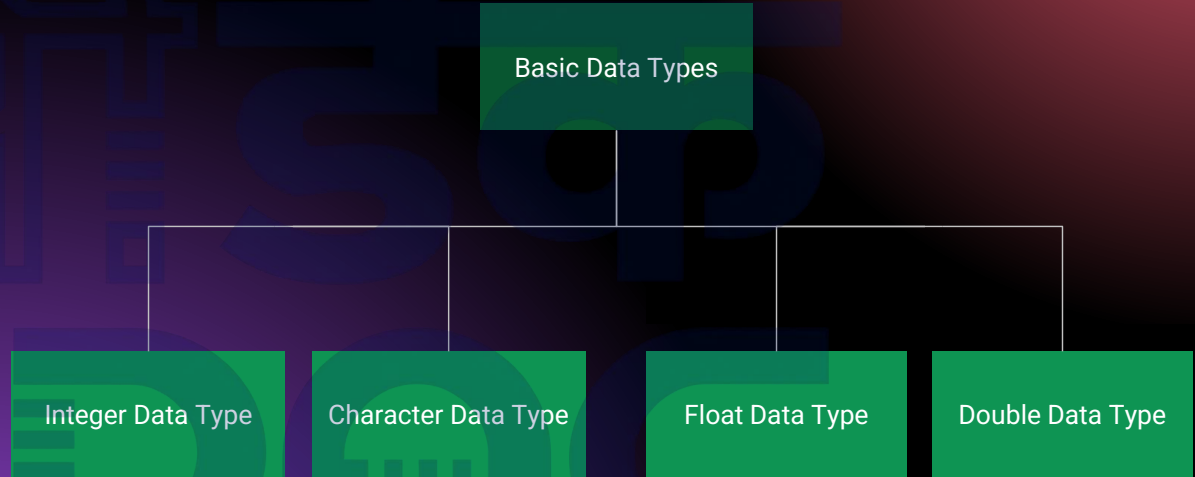


⇒ Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc.

⇒ Each data type requires different amounts of memory and has some specific operations which can be performed over it.



Basic Data Types in C



Integer Data Types In C

- The integer datatype in C is used to store the integer numbers (any number including positive, negative and zero without decimal part).
- Octal values, hexadecimal values, and decimal values can be stored in int data type in C.
- Syntax of Integer :

We use int keyword to declare the integer variable:

```
int var_name;
```

Integer Data Types In C

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Character Data Type

- Character data type allows its variable to store only a single character.
- The size of the character is 1 byte. It is the most basic data type in C.
- It stores a single character and requires a single byte of memory in almost all compilers.
- Syntax of char

The char keyword is used to declare the variable of character type:

```
char var_name;
```

Float Data Type

- In C programming float data type is used to store floating-point values.
- Float in C is used to store decimal and exponential values.
- Syntax of float

The float keyword is used to declare the variable as a floating point:

```
float var_name;
```

Double Data Type

- A Double data type in C is used to store decimal numbers (numbers with floating point values) with double precision.
- It is used to define numeric values which hold numbers with decimal values in C.
- The double data type is basically a precision sort of data type that is capable of holding 64 bits of decimal numbers or floating points. Since double has more precision as compared to that float.
- Syntax of Double

The variable can be declared as double precision floating point using the double keyword:

```
double var_name
```

Double Data Type

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

The void Data Type in C

The void type specifies that no value is available. It is used in three kinds of situations –

Function returns as void

There are various functions in C that do not return any value or you can say they return void.

A function with no return value has the return type as void.

For example, `void exit (int status);`

Function arguments as void

There are various functions in C which do not accept any parameter.

A function with no parameter can accept a void.

For example, `int rand(void);`

Pointers to void

A pointer of type `void *` represents the address of an object, but not its type.

For example, a memory allocation function `void *malloc(size_t size);` returns a pointer to void which can be casted to any data type.

Operators

- An expression in C is made up of operands , for eg $a = 2 + 3$ is a meaningful expression which involves 3 operands 2 , 3 , a and 2 operators i.e =,+ , Thus expression is sequence of Operators and operands
- Precedence of Operators:-Each Operator in C has a precedence associated with it , In a compound expression operator involved are of different precedence so operator with highest priority is evaluated first .
- Associativity : In compound expression when several operator are of same precedence operators are evaluated according to there associativity either left to right or right to left.

Classification of operators

- Unary Operators – Unary Operator operates on **only one operand** for example in the expression -3 – is a unary minus operator

Example : &, sizeof, !(logical negation), ~(bitwise negation), ++(increment), --(decrement) operator

- Binary Operators – Binary operator operates on **2 operands** for example expression 2-3, - acts as a binary minus operator as it operates on 2 operands 2 and 3

Example : *,/,<<(left shift),>>(Right shift),Logical And(&), Bitwise And(&)

- Ternary Operator – A ternary operator operates on three operands.

Example : Conditional operator (?:) is the only ternary operator in C

Classification Based on operator

Based on their role operators are classified as

01	Arithmetic Operators
02	Relational Operators
03	Logical Operators
04	Bitwise Operators
05	Assignment Operators

Arithmetic operators

- The C arithmetic operators are the symbols that are used to perform mathematical operations on operands. There are a total of
- 9 arithmetic operators in C to provide the basic arithmetic operations such as addition, subtraction, multiplication, etc.
- Types of Arithmetic Operators in C
- The C Arithmetic Operators are of two types based on the number of operands they work. These are as follows:
 - ◆ Binary Arithmetic Operators
 - ◆ Unary Arithmetic Operators

1. Binary Arithmetic Operators in C

The C binary arithmetic operators operate or work on two operands.

Operator	Name of the Operator	Arithmetic Operation	Syntax
+	Addition	Add two operands.	$x + y$
-	Subtraction	Subtract the second operand from the first operand.	$x - y$
*	Multiplication	Multiply two operands.	$x * y$
/	Division	Divide the first operand by the second operand.	x / y
%	Modulus	Calculate the remainder when the first operand is divided by the second operand.	$x \% y$

C program to demonstrate syntax of binary arithmetic operators

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 4, res;
```

```
    printf("a is %d and b is %d\n", a, b);
```

```
    // printing a and b
```

```
    res = a + b; // addition
```

```
    printf("a + b is %d\n", res);
```

```
    res = a - b; // subtraction
```

```
    res = a * b; // multiplication
```

```
    printf("a * b is %d\n", res);
```

```
    res = a / b; // division
```

```
    printf("a / b is %d\n", res);
```

```
    res = a % b; // modulus
```

```
    printf("a %% b is %d\n", res);
```

```
    return 0;
```

```
}
```

Output

a is 10 and b is 4

a + b is 14

a - b is 6

a * b is 40

a / b is 2

a % b is 2

2. Unary Arithmetic Operators in C

The unary arithmetic operators operate or work with a single operand.

Operator	Name of the Operator	Arithmetic Operation	Syntax
-	Decrement Operator	Decreases the integer value of the variable by one.	-h or h-
++	Increment Operator	Increases the integer value of the variable by one.	++h or h++
+	Unary Plus Operator	Returns the value of its operand.	+h
-	Unary Minus Operator	Returns the negative of the value of its operand.	-h

Increment Operator in C

The '++' operator is used to increment the value of an integer. It can be used in two ways

Pre-Increment

→ When placed **before the variable** name (also called the pre-increment operator), its value is incremented instantly.

→ Example:

```
a = ++x;
```

This example can be expanded to

```
a = (x = x + 1);
```

Post-Increment

→ When it is placed **after the variable name** (also called post-increment operator), its value is preserved temporarily until the execution of this statement and it **gets updated before the execution of the next statement.**

→ Example:

```
a = x++;
```

It can be expanded to

```
a = x;
```

```
x = x + 1;
```

Decrement Operator in C

The '-' operator is used to decrement the value of an integer. Just like the increment operator, the decrement operator can also be used in two ways

Pre-Decrement

- When placed before the variable name (also called the pre-decrement operator), its value is decremented instantly.
- Example:

```
a = -- x;
```

Post-Decrement

- When it is placed after the variable name (also called post-decrement operator), its value is preserved temporarily until the execution of this statement and it gets updated before the execution of the next statement.
- Example:

```
a = x- -;
```

Example of Unary Operators in C

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 5, b = 5, c = 5, d = 5;
```

```
    a++; // postfix increment
```

```
    ++b; // prefix increment
```

```
    c--; // postfix decrement
```

```
    --d; // prefix decrement
```

```
    printf("a = %d\n", a);
```

```
    printf("b = %d\n", b);
```

```
    printf("c = %d\n", c);
```

```
    printf("d = %d\n", d);
```

```
    return 0;
```

```
}
```

Output:

a = 6

b = 6

c = 4

d = 4

THANK YOU